UCI Network Target

Command Summary

Gideon Zweijtzer

All work Copyright © 2026 by
Gideon's Logic B.V.
All rights reserved.

Version 1.0, March 14, 2026

Table of Contents

1. Introduction **3**
1.1. Context 3
1.2. Purpose of this document 3
2. Commands **4**
2.1. NET_CMD_IDENTIFY (0x01) 4
2.2. NET_CMD_GET_INTERFACE_COUNT (0x02) 4
2.3. NET_CMD_GET_NETADDR (0x04) 4
2.4. NET_CMD_GET_IPADDR (0x05) 4
2.5. NET_CMD_SET_IPADDR (0x06) 5
2.6. NET_CMD_OPEN_TCP (0x07) 5
2.7. NET_CMD_OPEN_UDP (0x08) 5
2.8. NET_CMD_CLOSE_SOCKET (0x09) 5
2.9. NET_CMD_READ_SOCKET (0x10) 6
2.10. NET_CMD_WRITE_SOCKET (0x11) 6

1. Introduction

1.1. Context

The “Network Target” exposes the network stack in the Ultimate management application to be used by programs written on the C64 side. This interface offloads the responsibility of setting up and managing a TCP connection to a named host, allowing the C64 program to directly exchange data with external servers.

The “Network Target” is a target of the “Ultimate Command Interface” (UCI), and is thus accessible from the cartridge port, through some I/O registers. The document “Ultimate Command Interface – Register API” describes how commands are sent over this interface.

1.2. Purpose of this document

This document describes the commands that can be sent to this target, and their expected behavior and response. These commands are used to set up and close connections, and write and read from the network sockets.

2. Commands

In the Ultimate products and the Commodore 64 Ultimate, the "Network Target" is accessible through target $03. This shall be the first byte of the command.

The following paragraphs describe each of the commands of "Network target".

2.1. NET_CMD_IDENTIFY (0x01)

Command format: `$03 $01`

The "Identify" command sends back an identification string, such as: `"ULTIMATE-II NETWORK INTERFACE V1.0"`. The user software can use this function to query which targets exist, or to obtain version information.

The status channel will report `00,OK`, as this command cannot fail.

To avoid repetition, all commands may return `81,INVALID PARAMS` if the command length or format is incorrect.

2.2. NET_CMD_GET_INTERFACE_COUNT (0x02)

Command format: `$03 $02`

This command queries the Ultimate application for the number of available network interfaces (e.g., Ethernet, Wi-Fi).

Response: A single byte in the data channel representing the count of interfaces.

Status: `00,OK`.

2.3. NET_CMD_GET_NETADDR (0x04)

Command format: `$03 $04 <INTERFACE_INDEX>`

Retrieves the physical Hardware Address (MAC address) of the specified network interface.

Parameters: `<INTERFACE_INDEX>` is a single byte.

Response: 6 bytes representing the MAC address.

Status: `00,OK`, or `82,PARAMETER(S) OUT OF RANGE` if the index is invalid.

2.4. NET_CMD_GET_IPADDR (0x05)

Command format: `$03 $05 <INTERFACE_INDEX>`

The "Get IPAddr" command retrieves the current IP address assigned to the specified network interface.

Parameters: `<INTERFACE_INDEX>` is a single byte.

Response: 12 bytes representing the IP addresses in raw binary format.

- The first 4 bytes contain the used IP address, e.g. 192.168.1.146
- The second 4 bytes contain the local net mask, e.g. 255.255.255.0
- The last 4 bytes contain the local LAN gateway, e.g. 192.168.1.254

Status: `00,OK`.

2.5. NET_CMD_SET_IPADDR (0x06)

Command format: `$03 $06 <INTERFACE_INDEX> <IP_DATA>`

This command manually sets the IP address for a specific interface.

Parameters: Requires a 1-byte index followed by 12 bytes of IP data.

- The first 4 bytes shall contain the used IP address, e.g. 192.168.1.146
- The second 4 bytes shall contain the local net mask, e.g. 255.255.255.0
- The last 4 bytes shall contain the local LAN gateway, e.g. 192.168.1.254

Status: `00,OK`.

2.6. NET_CMD_OPEN_TCP (0x07)

Command format: `$03 $07 <PORT_LSB> <PORT_MSB> <HOSTNAME/IP_STRING>`

Opens a stream-based TCP connection to a remote host.

Parameters: The port is a 16-bit value (Little-Endian). The hostname is a null-terminated string or IP address.

Response: A single byte representing the **Socket Handle**.

Status: `00,OK`, or `84,UNRESOLVED HOST` if DNS lookup fails.

2.7. NET_CMD_OPEN_UDP (0x08)

Command format: `$03 $08 <PORT_LSB> <PORT_MSB> <HOSTNAME/IP_STRING>`

Opens a connectionless UDP socket for datagram exchange. Identical in format to OPEN_TCP, but initializes a SOCK_DGRAM type socket.

Response: A single byte representing the **Socket Handle**.

2.8. NET_CMD_CLOSE_SOCKET (0x09)

Command format: `$03 $09 <SOCKET_HANDLE>`

Closes the specified socket and frees the associated internal resources.

Status: `00,OK`.

2.9. NET_CMD_READ_SOCKET (0x10)

Command format: `$03 $10 <SOCKET_HANDLE> <LEN_LSB> <LEN_MSB>`

With this command a block of data can be read from an open socket.

Parameters: The length requested is a 16-bit value (Little-Endian).

Response: The data channel begins with a 2-byte header indicating the actual number of bytes read, LSB first, followed immediately by the data payload.

Status: `00,OK`, or `01,CONNECTION CLOSED BY HOST` if the remote side terminated the session.

2.10. NET_CMD_WRITE_SOCKET (0x11)

Command format: `$03 $11 <SOCKET_HANDLE> <DATA...>`

With this command, data can be sent through an open socket.

Parameters: The data to be sent follows the handle. The command length determines the payload size.

Response: A 2-byte value indicating the number of bytes successfully sent, LSB first.

Status: `00,OK`.